



AROR UNIVERSITY
OF ART, ARCHITECTURE,
DESIGN & HERITAGE,
SUKKUR, SINDH

Department of Artificial Intelligence & Multimedia Gaming
CSC-207: Database Systems

Lab # 05: To Work with SQL Operators & Querying Database Tables

Objectives

1. WHERE clause
2. Arithmetic operators
3. Comparison operators
4. Logical operators
5. Concatenation Operator
6. SQL Operator Precedence

WHERE clause in SQL

- The WHERE clause is used to filter rows based on a specified condition.
- Syntax: SELECT columns FROM table WHERE condition;
- The condition is evaluated for each row, and only rows that satisfy the condition are included.
- The WHERE clause is used to filter records.
- The WHERE clause is used to extract only those records that fulfill a specified condition.

Syntax:

SELECT column1, column2, ... FROM table_name

WHERE condition;

Note: The WHERE clause is not only used in SELECT statement, it is also used in UPDATE, DELETE statement, etc.! (will learn in upcoming labs)

The following SQL statement selects all the employees from the FIRST_NAME "Ellen", in the "employees" table:

Syntax: SELECT column_list FROM table_name WHERE column_name comparison_operator value;

Example:

```
SELECT * FROM employees WHERE FIRST_NAME = 'Ellen';
```

If you want to get the opposite, the employees other than Ellen then query will be:

```
SELECT * FROM employees WHERE FIRST_NAME <> 'Ellen';
```

Also you can use "!=" at the place of "<>".

Text Field & Numeric Fields:

SQL requires single quotes around text values (most database systems will also allow double quotes).

However, numeric fields should not be enclosed in quotes.

Example: SELECT * FROM employees WHERE EMPLOYEE_ID = 103;

Comparison operators in MySQL

Comparison operators in MySQL are used to compare two values (a column value with another column or a constant). They are most commonly used in the WHERE clause to filter rows based on conditions.

Operators in the WHERE clause

Operator	Type	Description
=	Comparison	Equal
>	Comparison	Greater than
<	Comparison	Less than
>=	Comparison	Greater than or equal
<=	Comparison	Less than or equal
<>	Comparison	Not equal. Note: In some versions of SQL this operator may be written as !=

AND	Logical	True when all conditions are true
OR	Logical	True when any condition is true
NOT	Logical	Reverses the result of a condition
BETWEEN	Logical	Between a certain range
LIKE	Logical	Search for a pattern
IN	Logical	To specify multiple possible values for a column

Example: 1. Display employees whose salary is greater than 10000.

```
SELECT employee_id, first_name, salary FROM employees WHERE salary > 10000;
```

2. Display employees whose salary is 8000 or more.

```
SELECT employee_id, first_name, salary FROM employees WHERE salary >= 8000;
```

3. Display employees whose job is not 'IT_PROG' and salary is less than 5000.

```
SELECT employee_id, first_name, job_id FROM employees WHERE job_id != 'IT_PROG' and salary <= 5000;
```

Syntax: AND, OR, NOT, BETWEEN, LIKE

```
SELECT column_list FROM table_name WHERE condition1 AND condition2;
```

```
SELECT column_list FROM table_name WHERE condition1 OR condition2;
```

```
SELECT column_list FROM table_name WHERE NOT condition;
```

```
SELECT column_list FROM table_name WHERE column_name BETWEEN value1 AND value2;
```

```
SELECT column_list FROM table_name WHERE column_name LIKE pattern;
```

Example: 1. Display employees who work in department 60 and have salary greater than 6000.

```
SELECT employee_id, first_name, department_id, salary FROM employees WHERE department_id = 60 AND salary > 6000;
```

2. Display employees who work in department 60 or department 90.

```
SELECT employee_id, first_name, department_id FROM employees WHERE department_id = 60 OR department_id = 90;
```

3. Display employees whose salary is between 5000 and 10000.

```
SELECT employee_id, first_name, salary FROM employees WHERE salary BETWEEN 5000 AND 10000;
```

4. Display employees whose first name starts with 'S'.

```
SELECT employee_id, first_name FROM employees WHERE first_name LIKE 'S%';
```

Common pattern symbols:

- % → any number of characters
- _ → exactly one character

Example: names with second letter = 'a':

```
SELECT employee_id, first_name FROM employees WHERE first_name LIKE '_a%';
```

5. Display employees working in departments 10, 20, or 30.

```
SELECT employee_id, first_name, department_id FROM employees WHERE department_id IN (10, 20, 30);
```

6. Display employees who are in department 60 or 90, and salary between 7000 and 12000.

```
SELECT employee_id, first_name, department_id, salary FROM employees WHERE (department_id IN (60, 90)) AND salary BETWEEN 7000 AND 12000;
```

Arithmetic operators in MySQL

Arithmetic operators in MySQL are used to perform mathematical calculations on numeric values or numeric columns in a table.

+	<i>Add</i>
-	<i>Subtract</i>
*	<i>Multiply</i>
/	<i>Divide</i>
%	<i>Modulo</i>

```
SELECT column1 operator column2, column_name operator number AS alias_name AS alias_name FROM table_name;
```

Example: 1. Add a fixed bonus of 1000 to each employee's salary.

```
SELECT employee_id, first_name, salary,  
       salary + 1000 AS salary_with_bonus
```

FROM employees;

2. Assume 30 working days and calculate daily salary.

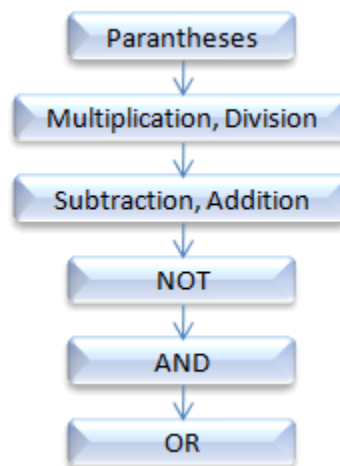
```
SELECT employee_id, first_name, salary, salary / 30 AS daily_salary FROM employees;
```

3. Calculate annual salary and annual salary after adding a 10% increment.

```
SELECT employee_id, first_name, salary, salary * 12 AS annual_salary, (salary * 12) + (salary * 12 * 0.10) AS annual_salary_after_increment FROM employees;
```

SQL Operator Precedence

- Operator precedence describes the order in which operations are performed when an expression is evaluated.
- Operations with a higher precedence are performed before those with a lower precedence.
- Parentheses has the highest precedence and OR has the lowest.



Exercises (Class)

Add here all the tasks performed in lab.

Exercises (Weekly)

1. Write a query to display EMPLOYEE_ID, FIRST_NAME, and SALARY of employees whose SALARY is less than \$3000.
2. Write a query to display FIRST_NAME, LASTNAME of all employees whose first name

starts with letter 'A'.

3. Write a query to display FIRST_NAME, JOB_ID, DEPARTMENT_ID of employees who are either PU_CLERK or belongs to MANAGER_ID = 114.
4. Write a query to display EMPLOYEE_ID, FIRST_NAME, and SALARY of employees whose salaries lies in the range of \$1500 to \$3000;
5. Write a query to display EMPLOYEE_ID, FIRST_NAME, and SALARY of employees whose commission is empty.
6. Write a query to display first names of all employees that end with alphabet 'N'.
7. Write a query to display FIRST_NAME, JOB_ID, DEPARTMENT_ID of employees who are not PU_CLERK.
8. Write a query to display EMPLOYEE_ID, FIRST_NAME, and SALARY of those employees who do not have salaries of \$3300, \$3200, \$2200.
9. Write a query to display names of those employees whose first name starts with 'A' and ends with 'N'.
10. Write a query to display the list of employee names that have letters 'LA' in their names.
11. Write a query to display the EMPLOYEE_ID, FIRST_NAME, and SALARY of employees. In that, the highest paid employee should display first and lowest paid should display last.
12. Write a query to display FIRST_NAME of employees that have "a" in the second position.
13. Write a query to display EMPLOYEE_ID, FIRST_NAME, and SALARY of employees whose salaries do not lies in the range of \$1500 to \$3000;
14. Write a query to display FIRST_NAME, LAST_NAME and DEPARTMENT_ID of all employees in departments 30 or 100 in ascending order.
15. Write a query to display FIRST_NAME, LAST_NAME and SALARY for all employees whose salary is not in the range \$10,000 through \$15,000 and are in department 30 or 100.
16. Write a query to display FIRST_NAME, LAST_NAME and HIRE_DATE for all employees who were hired in 1987.
17. Write a query to display the LAST_NAME of employees whose LAST_NAME have exactly 6 characters.
18. Write a query to display FIRST_NAME, SALARY and PF (15% of salary) of all employees.
19. Write a query to display FIRST_NAME, SALARY and commission amount (% of salary) of all employees.
20. Write a query to display FIRST_NAME, SALARY and NET_SALARY after 500 deduction from salary of all employees;